Senior Thesis

Collin Guedel

06 March, 2026

Abstract

Human land use, primarily through farming and its resulting land disturbances impacts the soil sink of atmospheric hydrogen maintained by soil microbial communities; this thesis' aim is to assess, both through measurements of soil H2 uptake rate and bacterial genomics, what governs this rate. Based upon preliminary agricultural data that I assess this summer, it appears that tilling has a significant impact on agricultural soils' uptake ; I would like to study this further by comparing farming practices and the resultant microbial behaviors in comparison to undisturbed, non-agriculture soils.

Contents

Introduction 2
Methods 2
Soy - Undisturbed 2
Soy - Disturbed Center 11
Results 19
Conclusion 19

References **20**

Appendix **20**

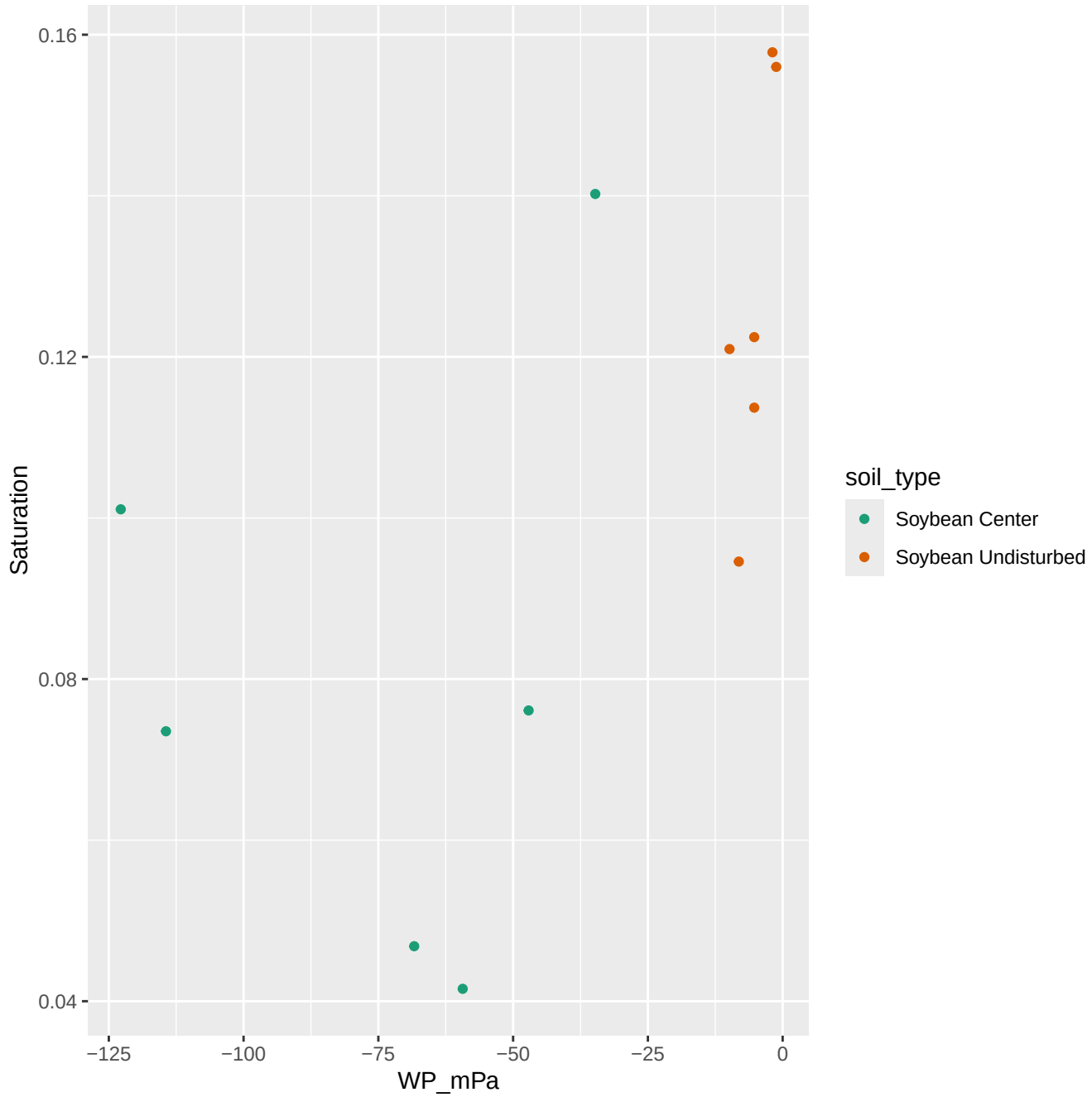
Introduction

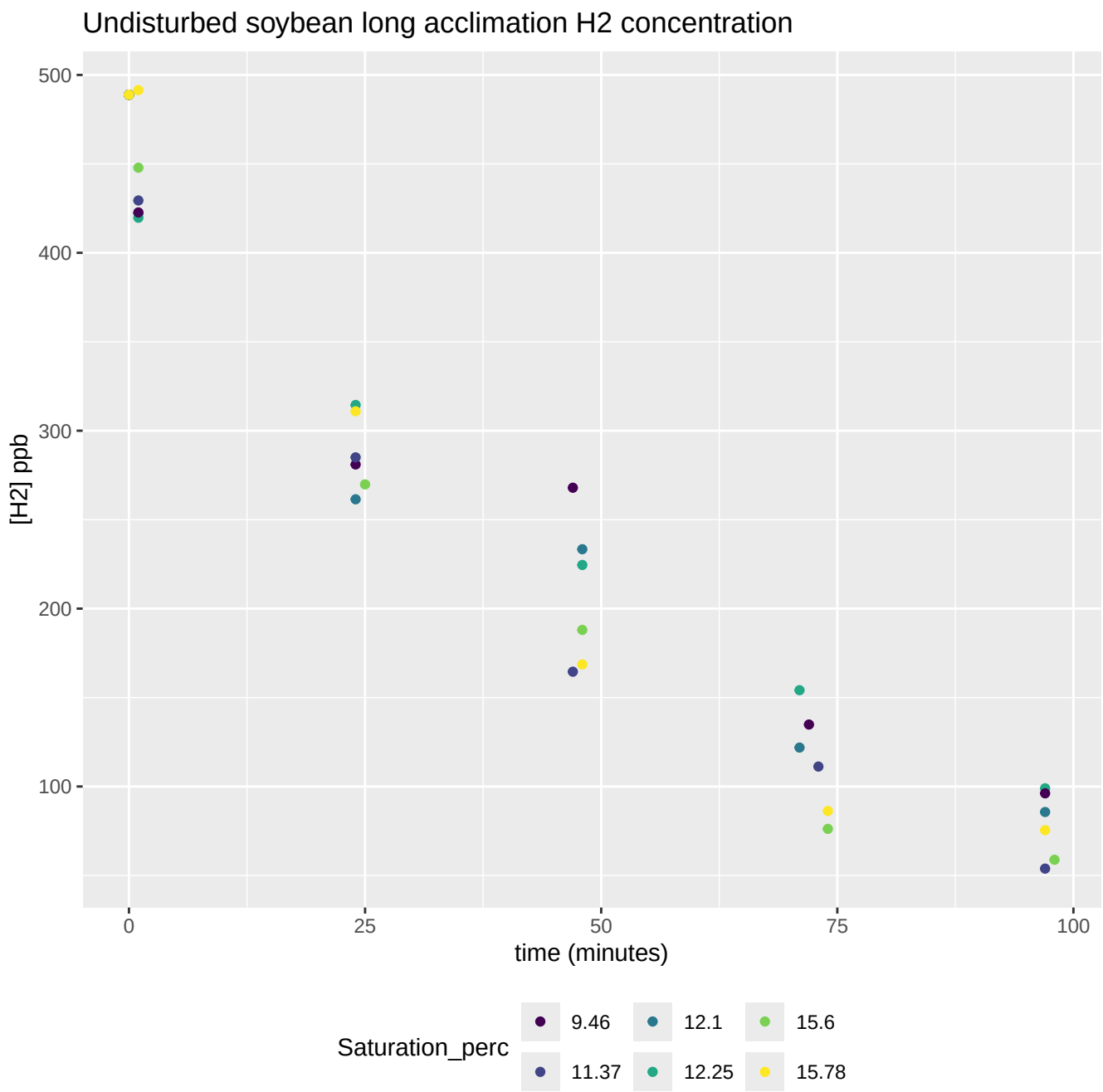
TBD Sand *et al.* (2023) says

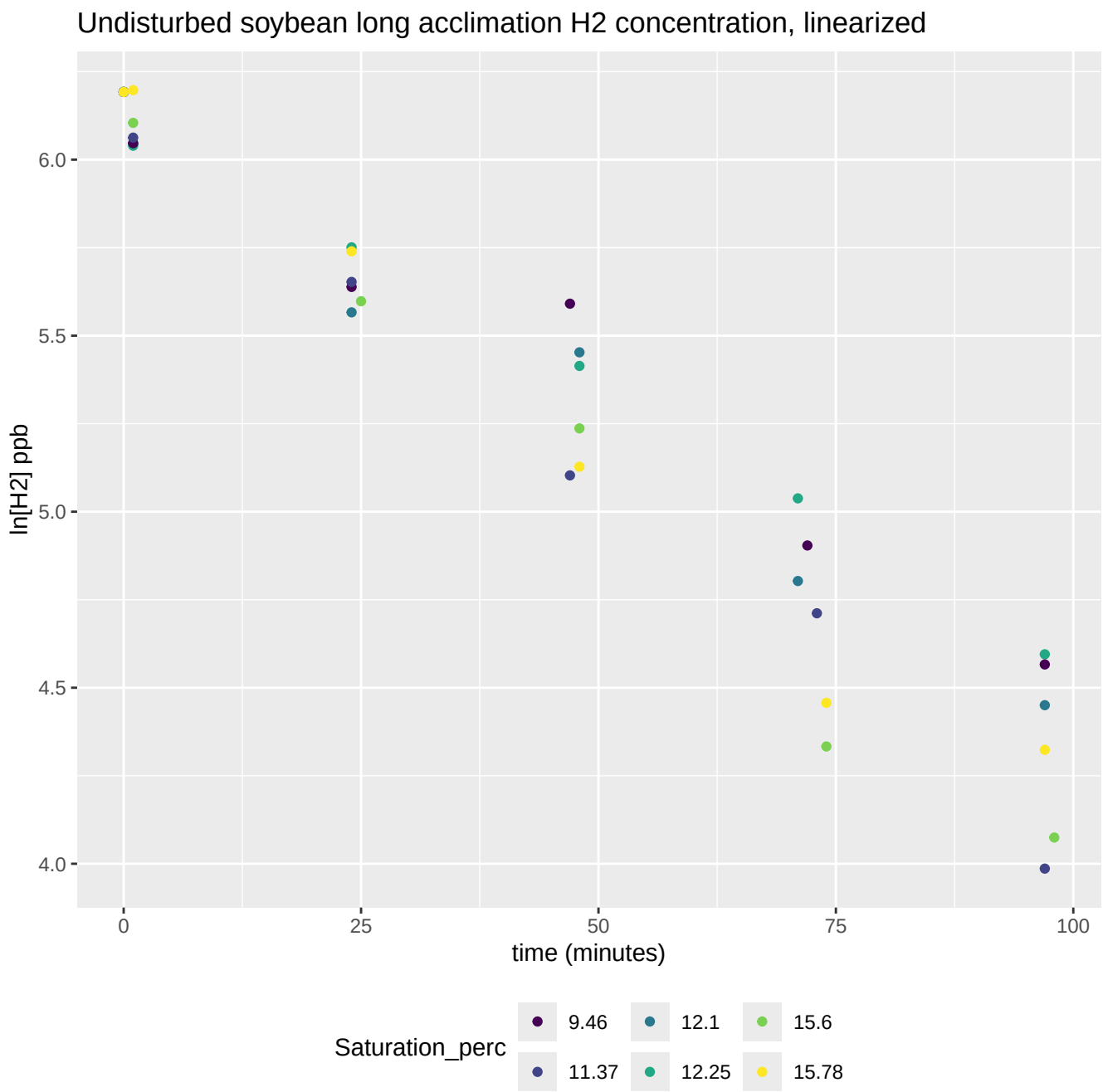
Methods

Soy - Undisturbed

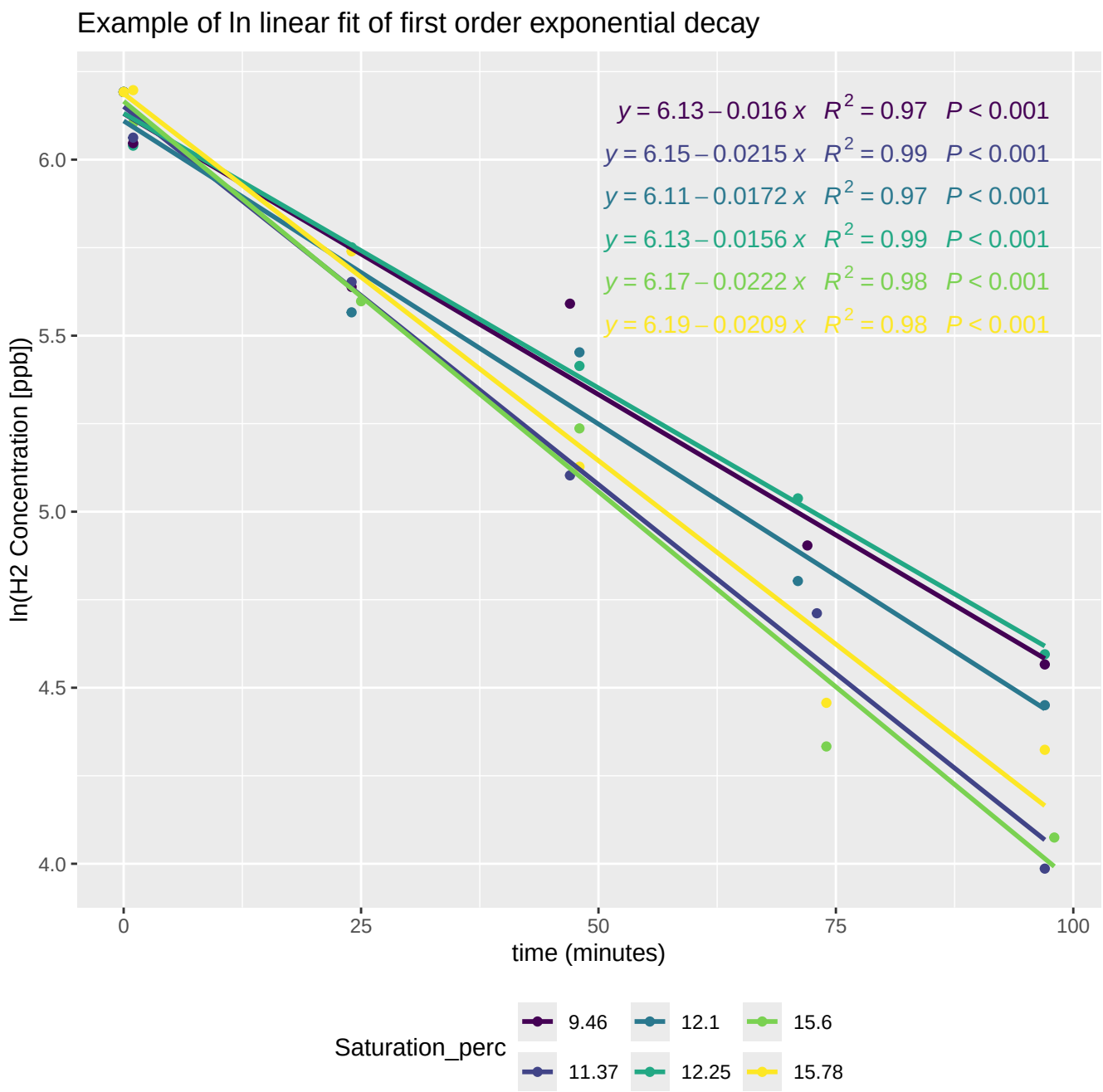
```
## Warning: Removed 6 rows containing missing values or values outside the scale range
## (`geom_point()`).
```


0.16
0.12
0.08
0.04
Saturation
−125
−100
−75
−50
−25
0
WP_mPa
soil_type
Soybean Center
Soybean Undisturbed


Undisturbed soybean long acclimation H2 concentration
500
400
300
200
100
[H2] ppb
0
25
50
75
100
time (minutes)
Saturation_perc
9.46
12.1
15.6
11.37
12.25
15.78

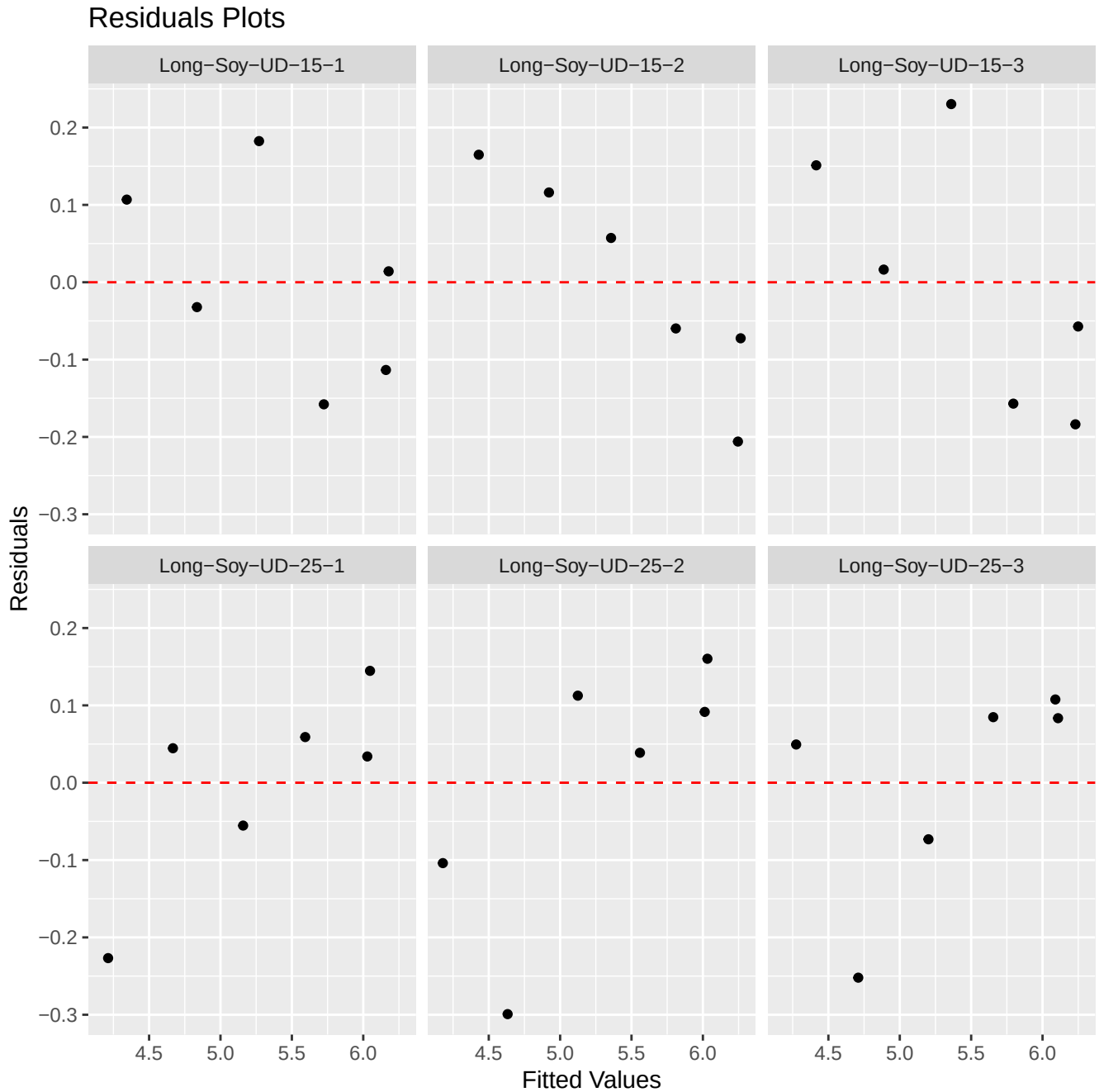



```
## `geom_smooth()` using formula = 'y ~ x'
```




```
## Warning: `fortify(<lm>)` was deprecated in ggplot2 4.0.0.
## i Please use `broom::augment(<lm>)` instead.
## i The deprecated feature was likely used in the ggplot2 package.
##   Please report the issue at <https://github.com/tidyverse/ggplot2/issues>.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
```

```
## generated.
```

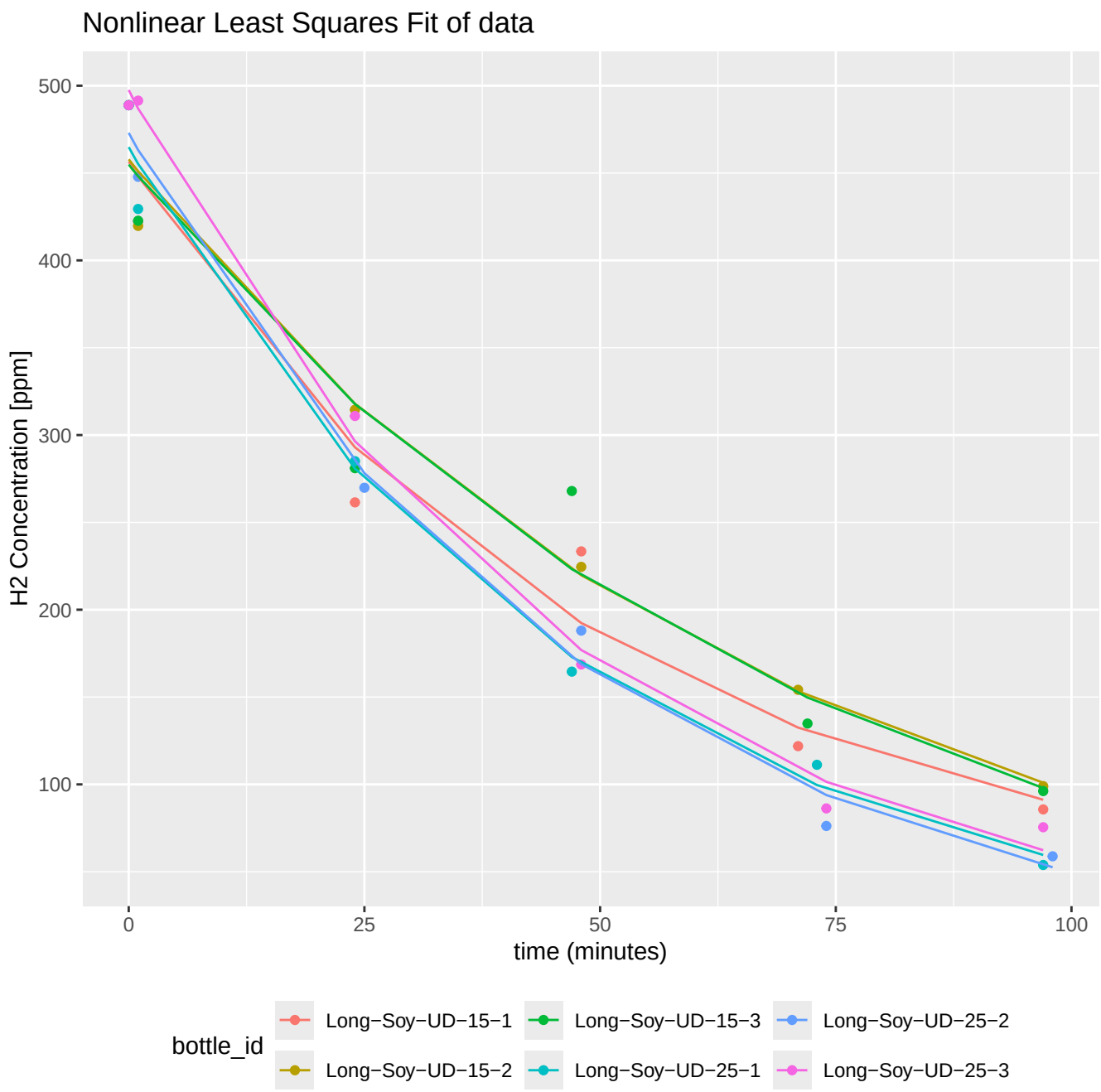


Of note: The fits are really good here, with very small residuals and R^2 values. The linearization seems to be effective without accounting for the asymptotic behavior near the tails, meaning that equilibrium is not sufficiently reached by the end of these incubation experiments within the time frame. I have included the fitting equations in the graph along

with the R^2 values - the p values don't mean much because they only compare these lines to a slope of 0, which is obviously out of the question.

```
## Adding missing grouping variables: `bottle_id`
```

```
## Warning: `qplot()` was deprecated in ggplot2 3.4.0.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

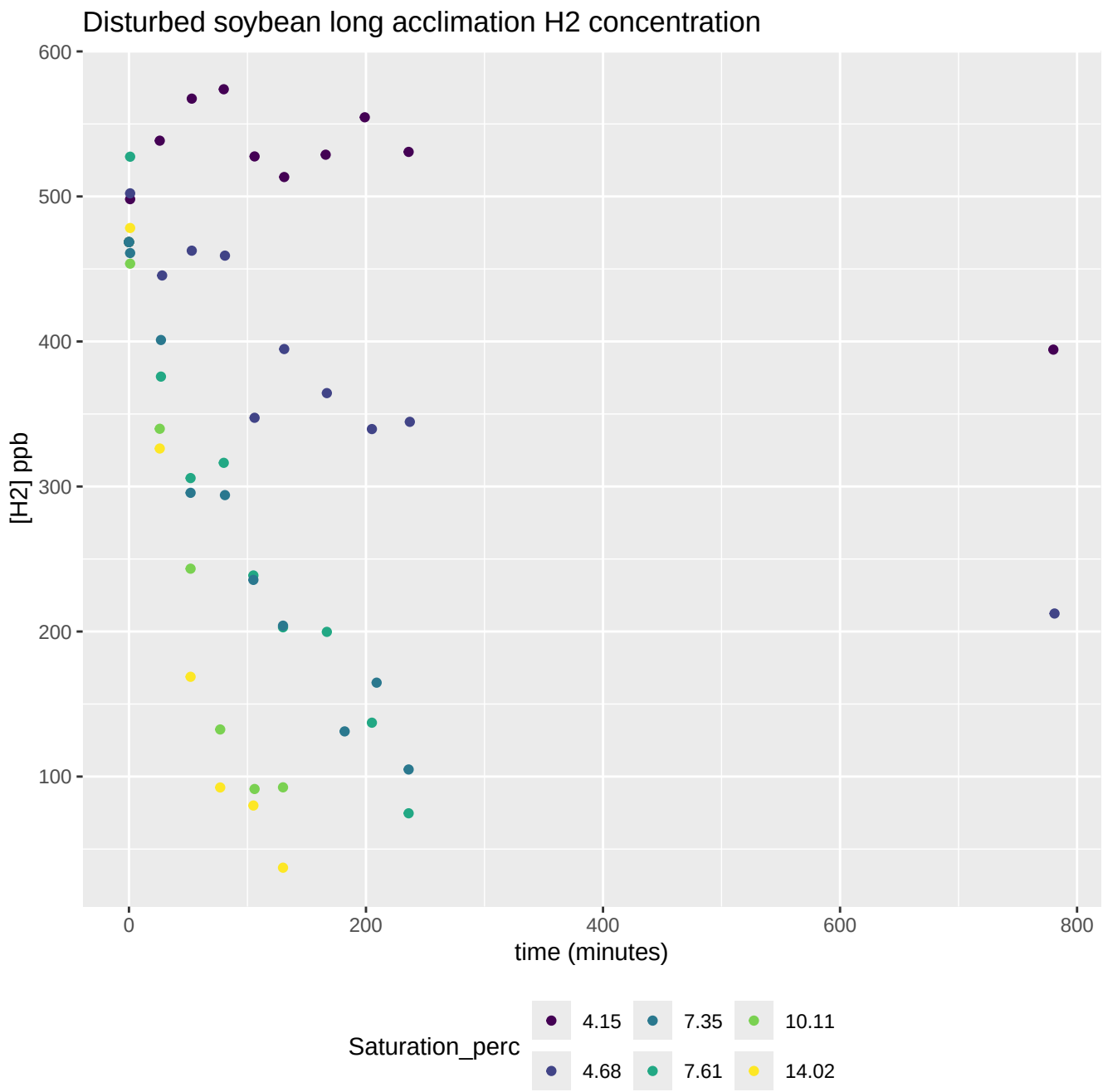


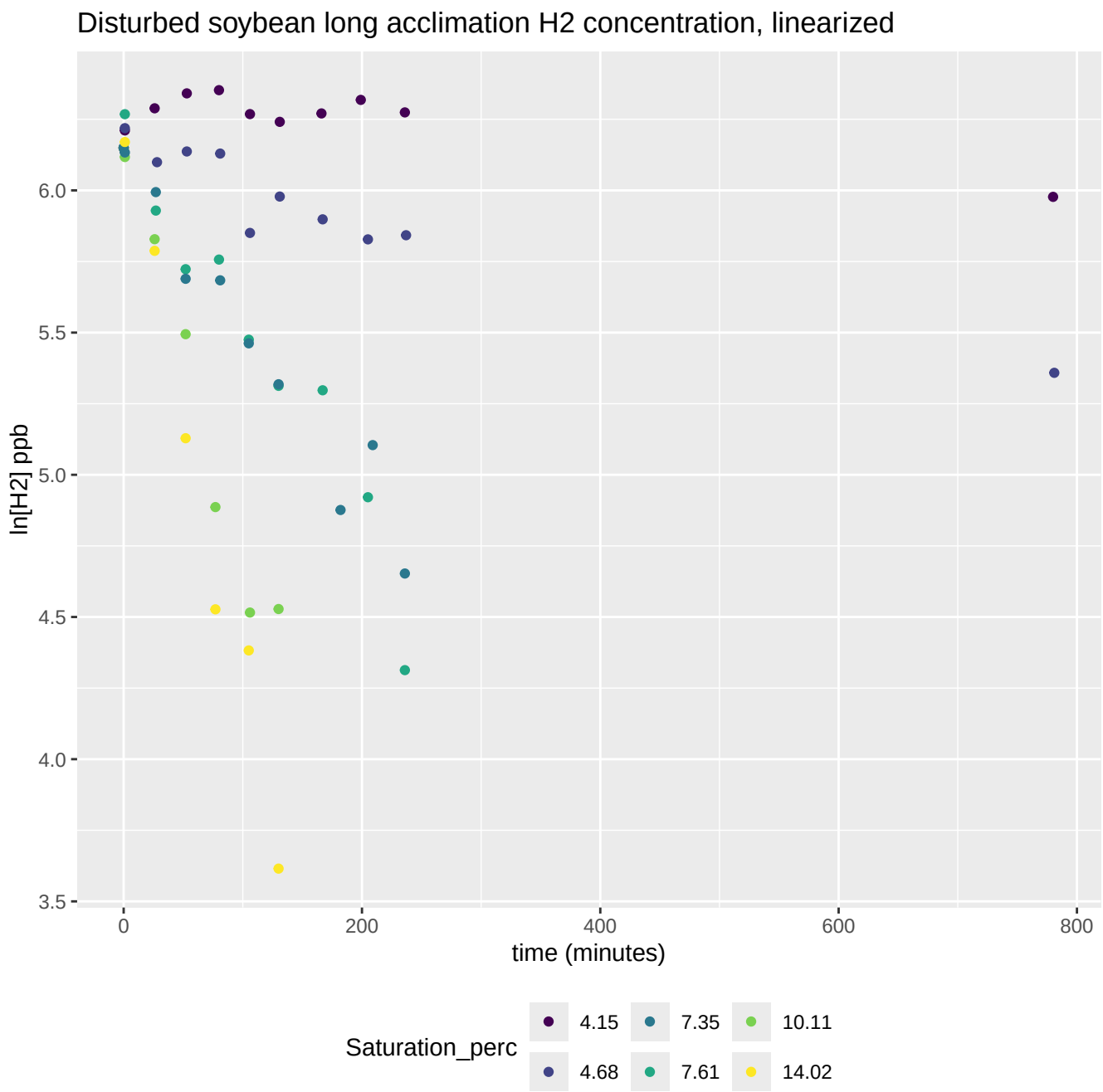
This nonlinear fit is less interpretable than the linearized fits seen above, because, as the data tables show, it is often guessing that the asymptote line is below 0 ppb of H2, which is impossible. The earlier linearized fitting methods make more physical sense and don't seem to diverge from linearity based upon the residuals plot, but they *may* seem to diverge from homoscedasticity - the residuals plots for each don't seem totally amorphous, but they may be a result of the very small sample sizes per bottle. Regardless, heteroscedasticity does not

introduce bias into the model (it doesn't sway the regression coefficient one way or another), but it does mean our models are less precise than the high R^2 values would make them appear to be.

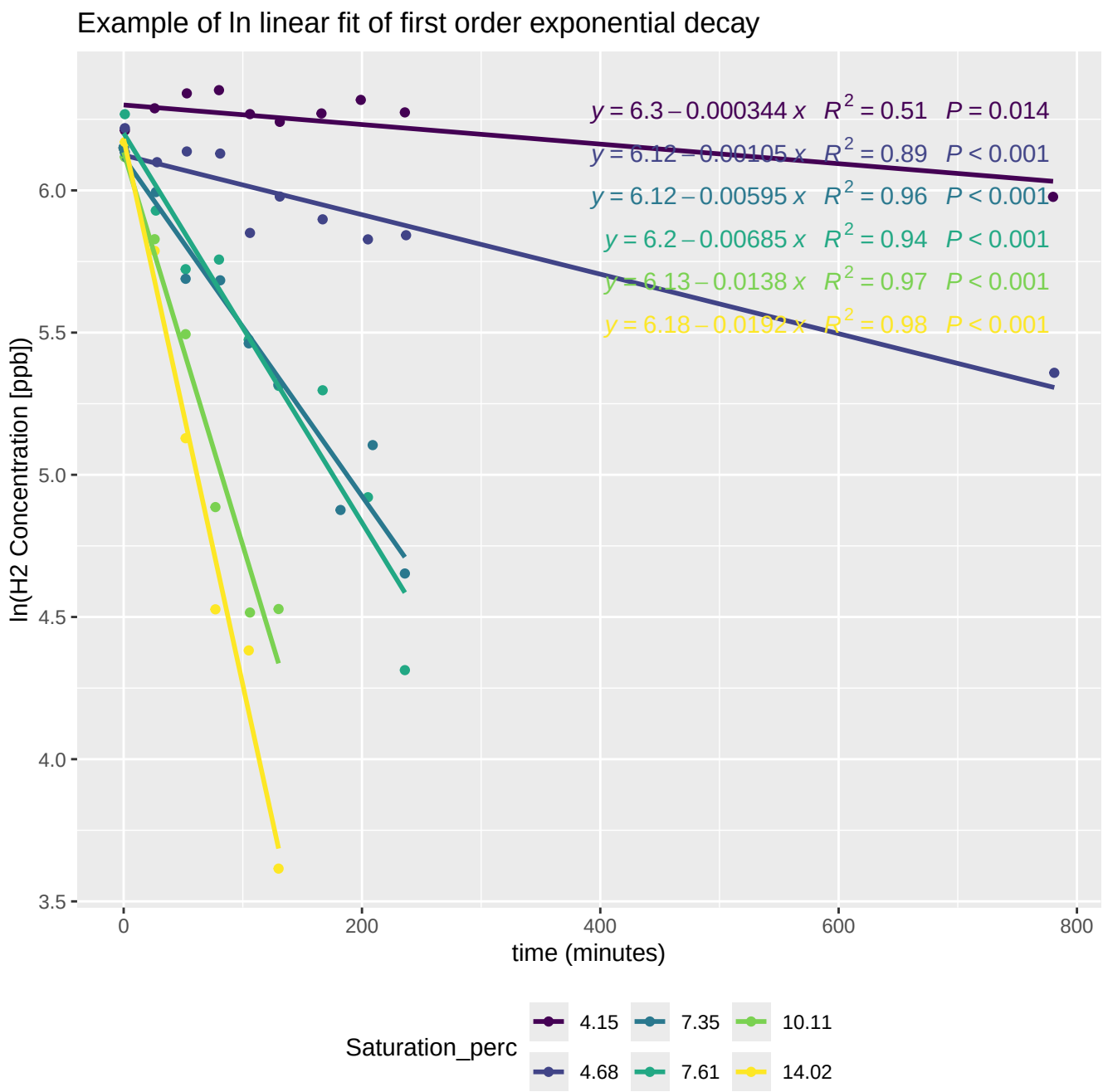
There are methods that I want to use to seek to address this - the first being a new-to-me method of regression called a robust regression, another being perhaps a background subtraction of the hydrogen background in the nitrogen carrier gas. Most importantly, I think more trial runs will show us how they are behaving at a grander scale and if there are systemic deviations from the assumed linearity of our natural log linearization.

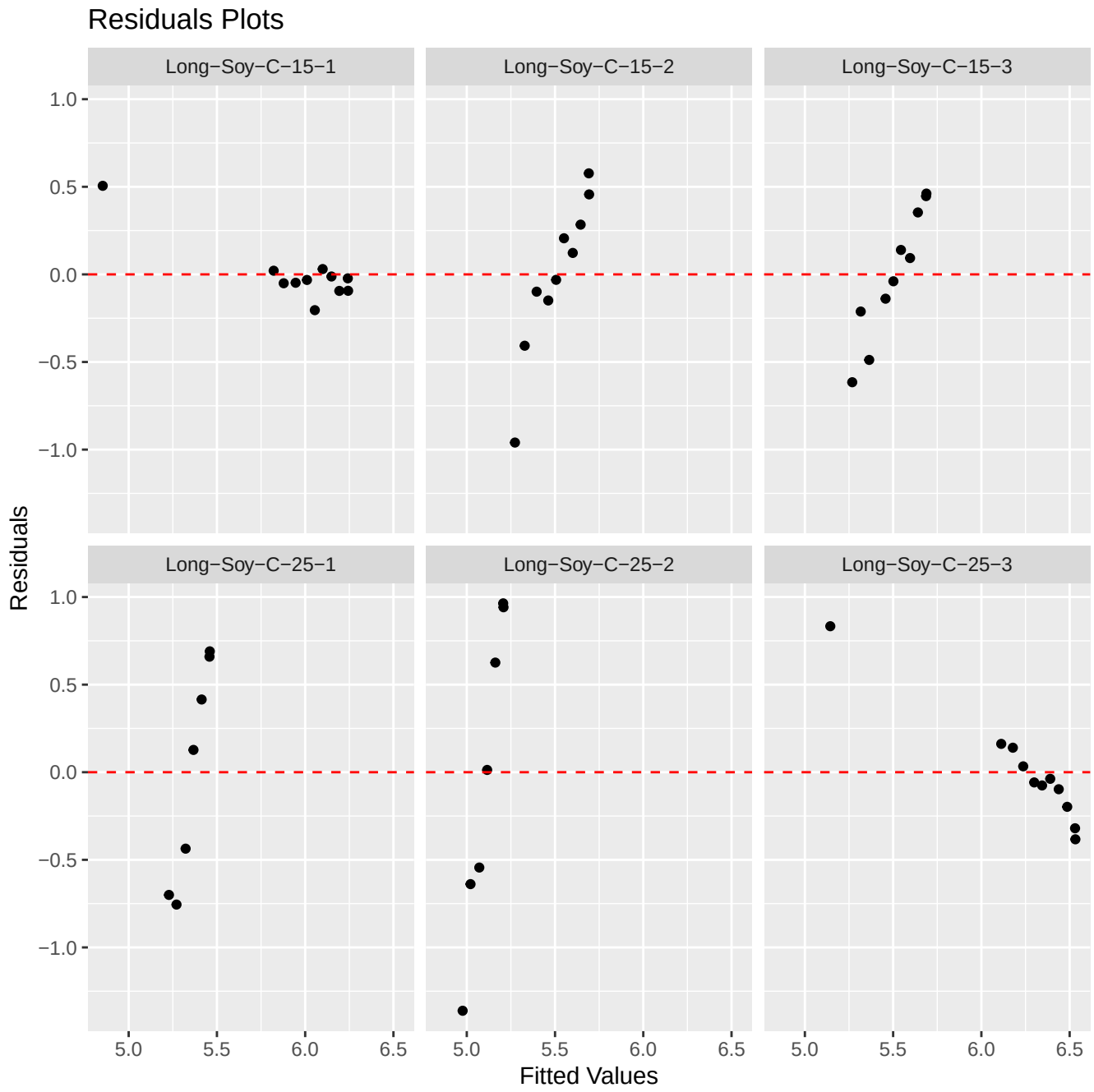
Soy - Disturbed Center



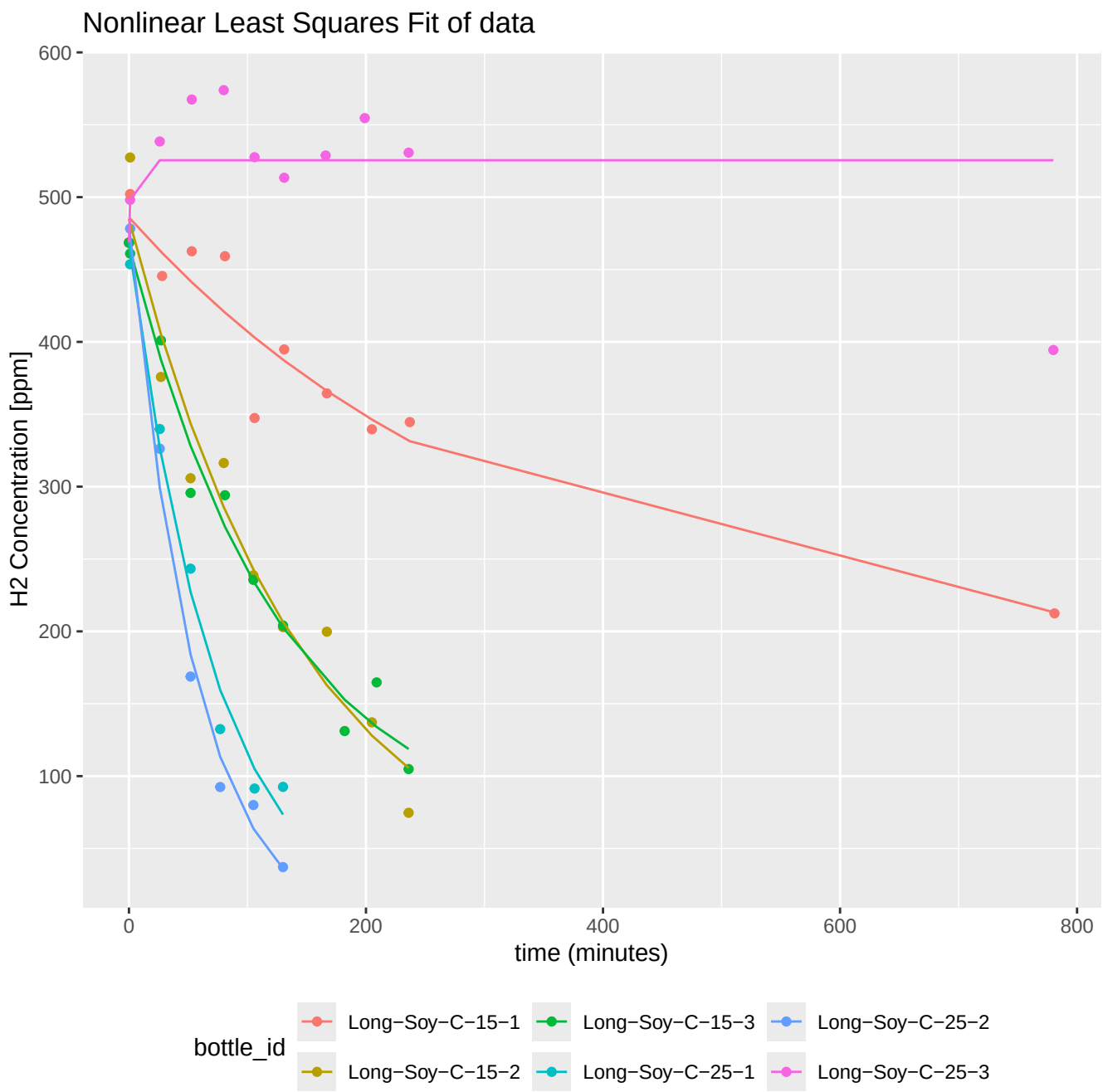



```
## `geom_smooth()` using formula = 'y ~ x'
```


Example of ln linear fit of first order exponential decay
y = 6.3 – 0.000344 x R2 = 0.51 P = 0.014
y = 6.12 – 0.00105 x R2 = 0.89 P < 0.001
y = 6.12 – 0.00595 x R2 = 0.96 P < 0.001
y = 6.2 – 0.00685 x R2 = 0.94 P < 0.001
y = 6.13 – 0.0138 x R2 = 0.97 P < 0.001
y = 6.18 – 0.0192 x R2 = 0.98 P < 0.001
ln(H2 Concentration [ppb])
6.0
5.5
5.0
4.5
4.0
3.5
0
200
400
600
800
time (minutes)
Saturation_perc
4.15
7.35
10.11
4.68
7.61
14.02



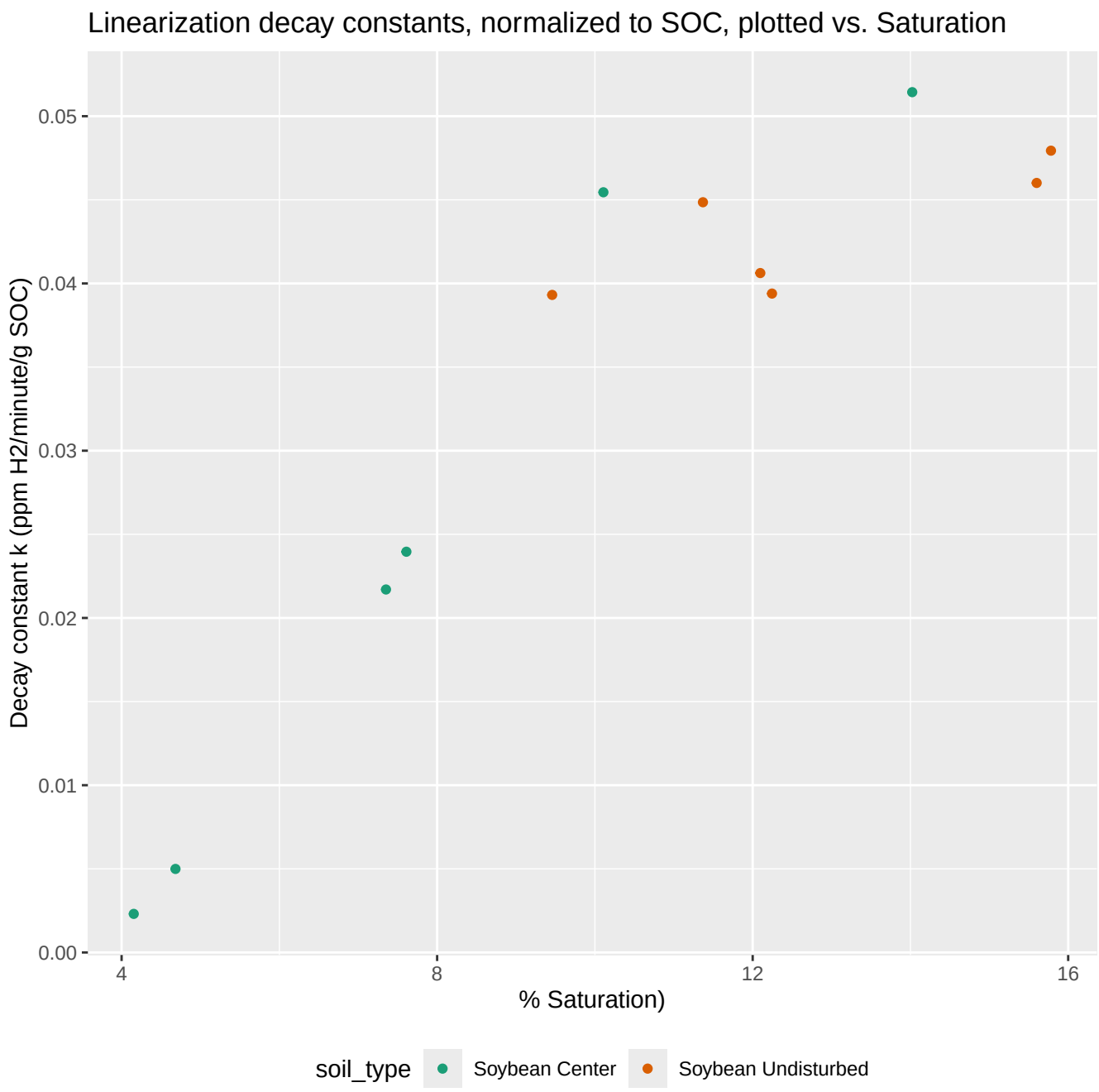

```
## Adding missing grouping variables: `bottle_id`
```




```
all_stats.df$k_norm = -1 *
↪  all_stats.df$time_offset_mins/(all_stats.df$`Mass bottle SOC (g)` *
   1000)
all_stats.df$alpha_norm = all_stats.df$nls_alpha/(all_stats.df$`Mass bottle
↪  SOC (g)`)
```

```r
ggplot(all_stats.df, mapping = aes(x = Saturation * 100, y = k_norm,
    color = soil_type)) + geom_point() + labs(title = "Linearization decay
    ↪  constants, normalized to SOC, plotted vs. Saturation") +
    xlab("% Saturation)") + ylab("Decay constant k (ppm H2/minute/g SOC)")
    ↪  +
    theme(legend.position = "bottom") + scale_color_brewer(palette =
    ↪  "Dark2")
```

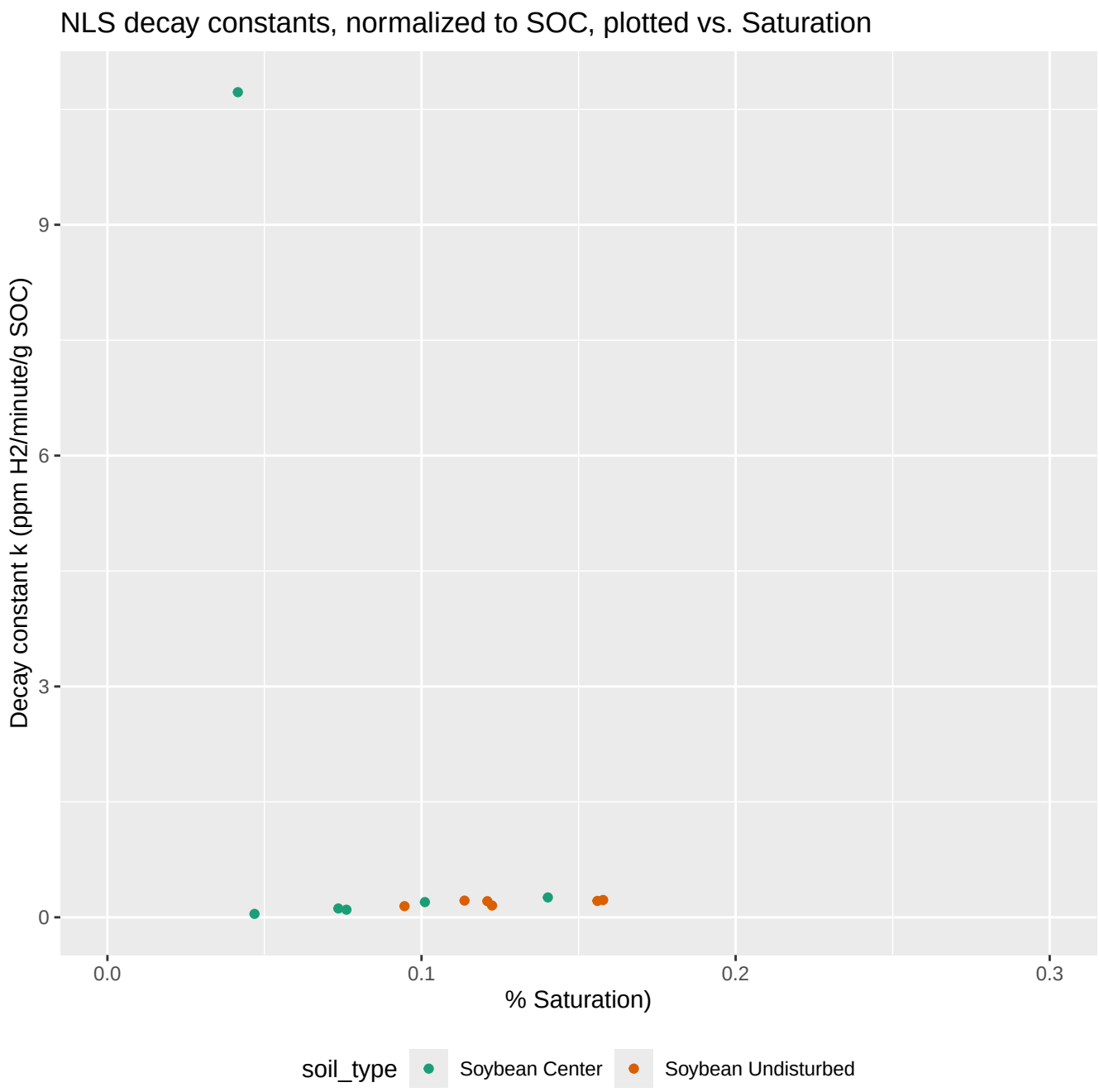
```
## Warning: Removed 6 rows containing missing values or values outside the scale range
## (`geom_point()`).
```




```
ggplot(all_stats.df, mapping = aes(x = Saturation, y = alpha_norm,
    color = soil_type)) + geom_point() + labs(title = "NLS decay constants,
    ↪  normalized to SOC, plotted vs. Saturation") +
    xlab("% Saturation)") + ylab("Decay constant k (ppm H2/minute/g SOC)")
    ↪  +
```

```r
    theme(legend.position = "bottom") + scale_color_brewer(palette =
    ↪  "Dark2") +
    xlim(0, 0.3)
```

```
## Warning: Removed 6 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



Results

Include draft work from CEE523 on equation writing.

Conclusion

References

Appendix

```r
rm(list = ls(all = TRUE))  # clear the workspace
options(width = 80)  # Sets the output width to roughly 80 characters
knitr::opts_chunk$set(tidy.opts = list(width.cutoff = 60), tidy = TRUE,
    echo = TRUE, warning = TRUE, message = TRUE, fig.align = "center",
    fig.height = 7, fig.width = 7)
library(installr)
library(formatR)
library(gridExtra)
library(ggpubr)
library(readxl)
library(ggtext)
library(scales)
library(tidyverse)
library(fitdistrplus)
library(actuar)
library(BSDA)
library(reshape2)
library(lmtest)
library(data.table)
library(broom)
library(ggpmisc)
library(bookdown)
library(usethis)

# theme_set(theme_minimal())
scale_color_brewer("spectral")
```

```r
scale_color_brewer(palette = "Dark2")

# updateR() setwd()
getwd()
df.ex_decay = read_excel("CEE523 Hydrogen Project Data.xlsx",
    sheet = "No_LitterAlightReplicate_1")
H_one_ppm = 584070.6667

df.ex_decay$H2_Conc_ppm = df.ex_decay$H2_area/H_one_ppm
H_eq_approx = min(df.ex_decay$H2_Conc_ppm)
H_eq_approx
df.ex_decay$ln_conc_origin = log(df.ex_decay$H2_Conc_ppm) -
↪  log(df.ex_decay$H2_Conc_ppm[1])
df.ex_decay$ln_conc_minus_eq_origin = -1 * log(df.ex_decay$H2_Conc_ppm -
    H_eq_approx)/log(df.ex_decay$H2_Conc_ppm[1] - H_eq_approx)
view(df.ex_decay)

long.df.ex_decay = melt(df.ex_decay, id.vars = c("Sampling_Time",
    "Time_diff_num", "time_point", "time_min", "injection_time",
    "time_diff_injection_and_sampling_min", "H2_area", "CO_Area",
    "H2_Conc_ppm"), variable.name = "type", value.name = "conc")
view(long.df.ex_decay)
mergeColumns <- function(df) {
    colstoKeep <- grep("\\.x$", names(all_stats.df), value = TRUE)
    colstoRemove <- grep("\\.y$", names(all_stats.df), value = TRUE)
    for (i in seq_along(colstoKeep)) {
        df[[colstoKeep[[i]]]][is.na(df[[colstoKeep[[i]]]])] <-
↪  df[[colstoRemove[[i]]]][is.na(df[[colstoKeep[[i]]]])]
        df[[colstoRemove[[i]]]] <- NULL
```

```r
        names(df)[names(df) %in% colstoKeep[[i]]] <- gsub(".x|.y",
            "", names(df)[names(df) %in% colstoKeep[[i]]])
    }
    return(df)
}
# Template: file <- 'C:/Users/crgue_ggna89k/OneDrive -
# Princeton University/Undergraduate Research/Thesis
# Files/Thesis Rmd
# Doc/2026-01-28/2026-01-28/2026-01-28.csv'; data <-
# read.csv(file)

import_rcp_csv <- function(file_path, date, n_compound) {
    # Note: file_path should be formatted like
    # 'C:/Users/crgue_ggna89k/OneDrive - Princeton
    # University/Undergraduate Research/Thesis Files/Thesis
    # Rmd Doc/2026-01-28/2026-01-28/2026-01-28.csv', except
    # EXCLUDE the dates format - simply include the
    # innermost folder that holds each of your date files
    # followed by a forward slash Express date as
    # 'year-month-day' so '2026-01-28'
    file = paste0(file_path, date, "/", date, "/", date, ".csv")

    n_cols <- max(count.fields(file, sep = ","))  #columns in spreadsheet
    n_cols
    max_compound = 6  # maximum number of compounds output by rcp

    # initial columns
    v.initial_names <- c("date_and_time", "run_mode",
↪   "analysis_buff_num_and_stream_num",
```

```r
    "gen_err_alarm", "lamp_v")

# template column names per compound
v.rep_names <- c("compound_name", "retention_time", "l_point_offset",
    "r_point_offset", "area", "rf", "processing_flag")
v.names = c(v.initial_names)
## repeat compound-specific, repeated columns (template
## lists 6 possible compounds)
for (i in 1:max_compound) {
    temp <- c()
    for (j in 1:length(v.rep_names)) {
        temp <- c(temp, paste0(v.rep_names[j], "_", as.character(i)))
    }
    v.names <- c(v.names, temp)
}
v.names <- c(v.names, "blank")  #correct for blank column at end

# data types of columns
remain_cols = n_cols - (length(v.initial_names)) - (length(v.rep_names)
↪ *
    n_compound)
v.col_classes <- c("character", rep(x = "integer", times = 4),
    rep(x = c("character", rep(x = "integer", times = 5),
        "character"), times = n_compound), if (remain_cols >
        0) {
        rep(x = "NULL", times = remain_cols)
    })

# check lengths
```

```r
    if (length(v.col_classes) != n_cols || length(v.names) !=
        n_cols || length(v.names) != length(v.col_classes)) {
        stop("Column counting error!")
    }

    data <- read.csv(file, header = F, colClasses = v.col_classes,
        col.names = v.names)
    # Make a date and time row for determining time
    # differences of samples
    for (i in 1:length(data)) {
        data$date_and_time[i] = paste0(substr(data$date_and_time[i],
            start = 1, stop = 13), ":", substr(data$date_and_time[i],
            start = 14, stop = 15))
    }
    data$date_and_time <- ymd_hm(data$date_and_time)
    return(data)
}

# file <- 'C:/Users/crgue_ggna89k/OneDrive - Princeton
# University/Undergraduate Research/Thesis Files/Thesis Rmd
# Doc/2026-01-28/2026-01-28/2026-01-28.csv' Generalized
# formula for any order of bottles - makes repeater above
# semi-obsolete
rcp_csv_preplot <- function(df, bottle_names, num_calib, num_air) {
    if (length(bottle_names) != (nrow(df) - num_calib - num_air)) {
        stop("number of bottle names does not equal number of rows in the
        ↪ data!")
    }
    bottling_time = 1  #about 30 seconds to seal each incubation bottle
```

```r
v_bottle_ml = 240  # volume of bottle
v_sample_ml = 2  # volume of gas sample and dilution

# sample head of dataframe for air and calibration
# hydrogen values

calib.v = head(df, num_calib)$area_1
air_H_avg = mean(df$area_1[(num_calib + 1):num_air])  # take average of initial air values

# remove initial values from dataframe (air baseline
# and standard calibration)
incubation.df = df[-(1:(num_air + num_calib)), ]  # excise initial calibration tests of air
rf1 = incubation.df$rf_1[1]  #set rf value as constant based upon initial rf value

if (length(calib.v) > 0) {
    calib_avg = mean(calib.v)  # take average of initial standard values
    rf_diff = calib_avg - rf1
    if (calib_avg != 0) {
        paste("The difference in the machine's rf value and this session's rf is",
            rf_diff)
        t.test(calib.v, mu = rf1)
    }
}
```

```r
# add bottle names to df of imported csv - make sure
# length of name vector = length of incubations

incubation.df$bottle_id = bottle_names

# Add time offset from start based upon bottling
# category
incubation.df$time_offset_mins

# time offset for bottles in general sequence Count
# number of unique bottle names
unique_bottle = unique(incubation.df$bottle_id)
n = length(unique_bottle)

# Trace through dataframe for each unique bottle,
# setting the initial index to the first occurrence of
# a given bottle - this is your time 0 for that bottle
for (i in 1:n) {
    first_occurrence_index_match <- match(unique_bottle[i],
        incubation.df$bottle_id)
    for (j in 1:nrow(incubation.df)) {
        # Now trace through the entire array searching
        # for every occurrence of this bottle
        # Subtracting out the time between the first
        # measurement and every subsequent measurement
        if (incubation.df$bottle_id[j] == unique_bottle[i]) {
            incubation.df$time_offset_mins[j] =
↪ as.numeric(difftime(incubation.df$date_and_time[j],
                incubation.df$date_and_time[first_occurrence_index_match],
```

```
                    units = "mins"), units = "mins") + bottling_time
            }
        }
    }

    # dilution correcting for added air to bottles
    incubation.df$ppb_dilution_corrected_1 =
↪   ((incubation.df$area_1/incubation.df$rf_1) *
        v_bottle_ml - ((air_H_avg/incubation.df$rf_1) *
↪   v_sample_ml))/(v_bottle_ml -
        v_sample_ml)

    # Adding rooted-at-air values to every bottle
    for (i in 1:n) {
        incubation.df <- add_row(incubation.df, ppb_dilution_corrected_1 =
↪   air_H_avg/rf1,
            time_offset_mins = 0, .before = 1, bottle_id = bottle_names[n +
                1 - i])
    }
    return(incubation.df)  # return cleaned and time-stepped output
    ↪  dataframe for plotting
}
meta.df <- read_excel("Farm Soils Data.xlsx", sheet = "Trial Metadata",
    col_names = T)

all_stats.df <- meta.df
ggplot(data = all_stats.df, mapping = aes(x = WP_mPa, y = Saturation,
    color = soil_type)) + geom_point() + scale_color_brewer(palette =
    ↪  "Dark2")
```

```r
file_path = paste0(getwd(), "/")
date = "2026-01-28"
n_compound = 2
bnames = rep(x = c("Long-Soy-UD-15-1", "Long-Soy-UD-15-2",
↪  "Long-Soy-UD-15-3",
    "Long-Soy-UD-25-1", "Long-Soy-UD-25-2", "Long-Soy-UD-25-3"),
    times = 5)
num.std = 0  # number of starter calibration samples
num.air = 5  # Number of starter air samples


import.df = import_rcp_csv(file_path, date, n_compound)  #import csv from
↪  above method

# Alter the RF because machine was being weird
altered_rf_1 = 533.515
import.df$rf_1 = rep(x = 533.515, times = nrow(import.df))

prepped.df = rcp_csv_preplot(import.df, bnames, num.std, num.air)
prepped.df$ppb_dilution_corrected_1_ln =
↪  log(prepped.df$ppb_dilution_corrected_1)

prepped_meta.df <- left_join(prepped.df, meta.df, by = "bottle_id")
prepped_meta.df <- mergeColumns(prepped_meta.df)
prepped_meta.df$Saturation_perc <- round(100 * prepped_meta.df$Saturation,
    digits = 2)
prepped_meta.df$Saturation_perc <- ordered(prepped_meta.df$Saturation_perc,
    levels = sort(unique(prepped_meta.df$Saturation_perc)))
```

```r
view(prepped_meta.df)


ggplot(prepped_meta.df, mapping = aes(x = time_offset_mins, y =
↪  ppb_dilution_corrected_1,
    color = Saturation_perc)) + geom_point() + labs(title = "Undisturbed
    ↪  soybean long acclimation H2 concentration") +
    xlab("time (minutes)") + ylab("[H2] ppb") + theme(legend.position =
    ↪  "bottom")

ggplot(prepped_meta.df, mapping = aes(x = time_offset_mins, y =
↪  ppb_dilution_corrected_1_ln,
    color = Saturation_perc)) + geom_point() + labs(title = "Undisturbed
    ↪  soybean long acclimation H2 concentration, linearized") +
    xlab("time (minutes)") + ylab("ln[H2] ppb") + theme(legend.position =
    ↪  "bottom")

ggplot(prepped_meta.df, mapping = aes(x = time_offset_mins, y =
↪  ppb_dilution_corrected_1_ln,
    color = Saturation_perc)) + geom_point() + geom_smooth(method = "lm",
    se = F) + stat_poly_eq(formula = y ~ x, aes(label =
    ↪  paste(after_stat(eq.label),
    after_stat(rr.label), after_stat(p.value.label), sep = "~~~")),
    parse = TRUE, label.x = "right") + labs(title = "Example of ln linear
    ↪  fit of first order exponential decay") +
    xlab("time (minutes)") + ylab("ln(H2 Concentration [ppb])") +
    theme(legend.position = "bottom")

# Residuals vs Fitted
```

```r
# Ensure 'bottle_id' is treated as a categorical variable
# (factor)
prepped_meta.df$bottle_id <- as.factor(prepped_meta.df$bottle_id)

model <- lm(ppb_dilution_corrected_1_ln ~ time_offset_mins +
    bottle_id, data = prepped_meta.df)
resid <- prepped_meta.df %>%
    mutate(fitted_values = fitted(model), residuals = residuals(model))
# aug_model <- augment(model)
ggplot(data = model, aes(x = .fitted, y = .resid)) + geom_point() +
    geom_hline(yintercept = 0, color = "red", linetype = "dashed") +
    labs(title = "Residuals Plots", x = "Fitted Values", y = "Residuals") +
    facet_wrap(~bottle_id)
# Define the model formula yf = asymptote, y0 = initial
# value, alpha = rate
decay_eq_root <- ppb_dilution_corrected_1 ~ yf + (air_H_avg/rf1 -
    yf) * exp(-alpha * time_offset_mins)
# fit <- nls( decay_eq_root data = prepped.df, start =
# list(yf = 50, alpha = 0.02) #requires a guess )

# Fit the model using nls() Provide starting guesses for
# parameters (for convergence) fit <- nls(decay_eq_mod,
# data = prepped.df, start = list(yf =
# min(prepped.df$ppb_dilution_corrected_1), y0 =
# max(data$ppb_dilution_corrected_1), alpha = 0.02)) Used
# website
# https://douglas-watson.github.io/post/2018-09_exponential_curve_fitting/
# for SS
```

```r
SelfStart = T

# Rooter values - can I make it self start? not
# self-starter (guess, but can root values)
if (SelfStart == F) {
    nls_fits_by_bottle <- prepped.df %>%
        group_by(bottle_id) %>%
        nest() %>%
        mutate(fit <- nls(decay_eq_mod, data = data, start = list(yf =
        ↪  min(data$value),
            y0 = max(data$value), alpha = 0.1)))
}

# linear fit (classical lm)
lm_fits_by_bottle <- prepped.df %>%
    group_by(bottle_id) %>%
    nest() %>%
    mutate(model = map(data, ~lm(ppb_dilution_corrected_1 ~
    ↪  time_offset_mins,
        data = .x)), results = map(model, glance), tidied = map(model,
        tidy))  # glance() for model fit statistics
lm_grouped_fit_stats <- lm_fits_by_bottle %>%
    unnest(results) %>%
    dplyr::select(bottle_id, r.squared, adj.r.squared, AIC, p.value)  #
↪  Select relevant columns
lm_tidied <- lm_fits_by_bottle %>%
    unnest(tidied) %>%
    dplyr::select(term, estimate) %>%
```

```r
    spread(term, estimate)

# Run selfstarting function (unrooted, no guessing)
nls_fits_by_bottle <- prepped.df %>%
    group_by(bottle_id) %>%
    nest() %>%
    mutate(fit = map(data, ~nls(ppb_dilution_corrected_1 ~
     ↪  SSasymp(time_offset_mins,
        yf, y0, log_alpha), data = ., start = c(yf = 0, y0 = 600,
        log_alpha = 0.001))), tidied = map(fit, tidy), augmented = map(fit,
        augment), coefs = map(fit, coef), results = map(fit,
        glance))
# tabular output
coefficients <- nls_fits_by_bottle %>%
    unnest(tidied) %>%
    dplyr::select(bottle_id, term, estimate) %>%
    spread(term, estimate) %>%
    mutate(nls_alpha = exp(log_alpha))
results <- nls_fits_by_bottle %>%
    dplyr::select(bottle_id, results) %>%
    unnest(results)
results <- results %>%
    rename(sigma_nls = sigma, isConv_nls = isConv, finTol_nls = finTol,
        logLik_nls = logLik, AIC_nls = AIC, BIC_nls = BIC, deviance_nls =
         ↪  deviance,
        df.residual_nls = df.residual, nobs_nls = nobs)

all_stats.df <- full_join(all_stats.df, lm_tidied, by = "bottle_id")
all_stats.df <- full_join(all_stats.df, lm_grouped_fit_stats,
```

```r
    by = "bottle_id")
all_stats.df <- full_join(all_stats.df, coefficients, by = "bottle_id")
all_stats.df <- full_join(all_stats.df, results, by = "bottle_id")
all_stats.df <- mergeColumns(all_stats.df)
view(all_stats.df)

# ggplot graph
augmented <- nls_fits_by_bottle %>%
    unnest(augmented)

qplot(time_offset_mins, ppb_dilution_corrected_1, data = augmented,
    geom = "point", colour = bottle_id) + geom_line(aes(y = .fitted)) +
    labs(title = "Nonlinear Least Squares Fit of data") + xlab("time
    ↪  (minutes)") +
    ylab("H2 Concentration [ppm]") + theme(legend.position = "bottom")

## IMPORTANT -RSE and AIC for these models!!! ANOVA too -
## part of this thesis can be to test model validity

## Self-Start Model example Example: Forcing 'a' to be 10
## in a SSmodel mySS <- selfStart( model = ~ a * exp(b *
## x), initial = function(mCall, data, LHS) { # Force a=10
## instead of estimating it value <- c(a = 10, b = 0.5)
## value }, parameters = c('a', 'b') ) EqRootSS <-
## selfStart( model = ~ yf + (y0 - yf) * exp(-k * t) )
file_path = paste0(getwd(), "/")
date = "2026-02-12"
n_compound = 2
bnames = c(rep(x = c("Long-Soy-C-25-1", "Long-Soy-C-25-2",
↪  "Long-Soy-C-25-3",
```

```
    "Long-Soy-C-15-1", "Long-Soy-C-15-2", "Long-Soy-C-15-3"),
    times = 6), rep(x = c("Long-Soy-C-25-3", "Long-Soy-C-15-1",
    "Long-Soy-C-15-2", "Long-Soy-C-15-3"), times = 3), "Long-Soy-C-25-3",
    "Long-Soy-C-15-1")
num.std = 3  # number of starter calibration samples
num.air = 5  # Number of starter air samples


import.df <- import_rcp_csv(file_path, date, n_compound)  #import csv from
↪  above method
import.df <- import.df[-nrow(import.df), ]

prepped.df = rcp_csv_preplot(import.df, bnames, num.std, num.air)
prepped.df$ppb_dilution_corrected_1_ln =
↪  log(prepped.df$ppb_dilution_corrected_1)

prepped_meta.df <- left_join(prepped.df, meta.df, by = "bottle_id")
prepped_meta.df <- mergeColumns(prepped_meta.df)
prepped_meta.df$Saturation_perc <- round(100 * prepped_meta.df$Saturation,
    digits = 2)
prepped_meta.df$Saturation_perc <- ordered(prepped_meta.df$Saturation_perc,
    levels = sort(unique(prepped_meta.df$Saturation_perc)))

view(prepped_meta.df)
ggplot(prepped_meta.df, mapping = aes(x = time_offset_mins, y =
↪  ppb_dilution_corrected_1,
    color = Saturation_perc)) + geom_point() + labs(title = "Disturbed
    ↪  soybean long acclimation H2 concentration") +
    xlab("time (minutes)") + ylab("[H2] ppb") + theme(legend.position =
    ↪  "bottom")
```

```r
ggplot(prepped_meta.df, mapping = aes(x = time_offset_mins, y =
↪  ppb_dilution_corrected_1_ln,
    color = Saturation_perc)) + geom_point() + labs(title = "Disturbed
    ↪  soybean long acclimation H2 concentration, linearized") +
    xlab("time (minutes)") + ylab("ln[H2] ppb") + theme(legend.position =
    ↪  "bottom")


ggplot(prepped_meta.df, mapping = aes(x = time_offset_mins, y =
↪  ppb_dilution_corrected_1_ln,
    color = Saturation_perc)) + geom_point() + geom_smooth(method = "lm",
    se = F) + stat_poly_eq(formula = y ~ x, aes(label =
    ↪  paste(after_stat(eq.label),
    after_stat(rr.label), after_stat(p.value.label), sep = "~~~")),
    parse = TRUE, label.x = "right") + labs(title = "Example of ln linear
    ↪  fit of first order exponential decay") +
    xlab("time (minutes)") + ylab("ln(H2 Concentration [ppb])") +
    theme(legend.position = "bottom")

# Residuals vs Fitted

# Ensure 'bottle_id' is treated as a categorical variable
# (factor)
prepped_meta.df$bottle_id <- as.factor(prepped_meta.df$bottle_id)

model <- lm(ppb_dilution_corrected_1_ln ~ time_offset_mins +
    bottle_id, data = prepped_meta.df)
second_test_resid <- prepped_meta.df %>%
```

```r
    mutate(fitted_values = fitted(model), residuals = residuals(model))
# aug_model <- augment(model)
ggplot(data = model, aes(x = .fitted, y = .resid)) + geom_point() +
    geom_hline(yintercept = 0, color = "red", linetype = "dashed") +
    labs(title = "Residuals Plots", x = "Fitted Values", y = "Residuals") +
    facet_wrap(~bottle_id)
# linear fit (classical lm)
lm_fits_by_bottle <- prepped.df %>%
    group_by(bottle_id) %>%
    nest() %>%
    mutate(model = map(data, ~lm(ppb_dilution_corrected_1 ~
    ↪  time_offset_mins,
        data = .x)), results = map(model, glance), tidied = map(model,
        tidy))
lm_grouped_fit_stats <- lm_fits_by_bottle %>%
    unnest(results) %>%
    dplyr::select(bottle_id, r.squared, adj.r.squared, AIC, p.value)
lm_tidied <- lm_fits_by_bottle %>%
    unnest(tidied) %>%
    dplyr::select(term, estimate) %>%
    spread(term, estimate)

# Run selfstarting function (unrooted, no guessing)
nls_fits_by_bottle <- prepped.df %>%
    group_by(bottle_id) %>%
    nest() %>%
    mutate(fit = map(data, ~nls(ppb_dilution_corrected_1 ~
    ↪  SSasymp(time_offset_mins,
        yf, y0, log_alpha), data = ., start = c(yf = 0, y0 = 700,
```

```
        log_alpha = 0.001))), tidied = map(fit, tidy), augmented = map(fit,
        augment), coefs = map(fit, coef), results = map(fit,
        glance))
# tabular output
coefficients <- nls_fits_by_bottle %>%
    unnest(tidied) %>%
    dplyr::select(bottle_id, term, estimate) %>%
    spread(term, estimate) %>%
    mutate(nls_alpha = exp(log_alpha))
results <- nls_fits_by_bottle %>%
    dplyr::select(bottle_id, results) %>%
    unnest(results)
results <- results %>%
    rename(sigma_nls = sigma, isConv_nls = isConv, finTol_nls = finTol,
        logLik_nls = logLik, AIC_nls = AIC, BIC_nls = BIC, deviance_nls =
        ↪  deviance,
        df.residual_nls = df.residual, nobs_nls = nobs)

all_stats.df <- full_join(all_stats.df, lm_tidied, by = "bottle_id")
all_stats.df <- full_join(all_stats.df, lm_grouped_fit_stats,
    by = "bottle_id")
all_stats.df <- full_join(all_stats.df, coefficients, by = "bottle_id")
all_stats.df <- full_join(all_stats.df, results, by = "bottle_id")
all_stats.df <- mergeColumns(all_stats.df)
view(all_stats.df)

augmented <- nls_fits_by_bottle %>%
    unnest(augmented)
```

```r
qplot(time_offset_mins, ppb_dilution_corrected_1, data = augmented,
    geom = "point", colour = bottle_id) + geom_line(aes(y = .fitted)) +
    labs(title = "Nonlinear Least Squares Fit of data") + xlab("time
    ↪  (minutes)") +
    ylab("H2 Concentration [ppm]") + theme(legend.position = "bottom")
all_stats.df$k_norm = -1 *
↪  all_stats.df$time_offset_mins/(all_stats.df$`Mass bottle SOC (g)` *
    1000)
all_stats.df$alpha_norm = all_stats.df$nls_alpha/(all_stats.df$`Mass bottle
↪  SOC (g)`)

ggplot(all_stats.df, mapping = aes(x = Saturation * 100, y = k_norm,
    color = soil_type)) + geom_point() + labs(title = "Linearization decay
    ↪  constants, normalized to SOC, plotted vs. Saturation") +
    xlab("% Saturation)") + ylab("Decay constant k (ppm H2/minute/g SOC)")
    ↪  +
    theme(legend.position = "bottom") + scale_color_brewer(palette =
    ↪  "Dark2")

ggplot(all_stats.df, mapping = aes(x = Saturation, y = alpha_norm,
    color = soil_type)) + geom_point() + labs(title = "NLS decay constants,
    ↪  normalized to SOC, plotted vs. Saturation") +
    xlab("% Saturation)") + ylab("Decay constant k (ppm H2/minute/g SOC)")
    ↪  +
    theme(legend.position = "bottom") + scale_color_brewer(palette =
    ↪  "Dark2") +
    xlim(0, 0.3)
```

```r
y0 = df.ex_decay$ln_conc_origin
y1 = long.df.ex_decay$conc[15:28]
x = df.ex_decay$time_min
model <- lm(y0 ~ x)
model <- lm(y1 ~ x)

lm(y ~ 1)  #explicit intercept is the only term

lm(y ~ x)  #implicit intercept
lm(y ~ 1 + x)  #explicit intercept

lm(y ~ -1 + x)  #no intercept
lm(y ~ 0 + x)  #no intercept

coef(model)  # the coefficients
fitted(model)  # y-hat values
summary(model)

names(model)
names(summary(model))
ggplot(df.ex_decay, mapping = aes(x = time_min, y = H2_Conc_ppm)) +
    geom_point() + labs(title = "Example of first order exponential decay")
    ↪  +
    xlab("time (minutes)") + ylab("H2 Concentration [ppm]") +
    theme(legend.position = "bottom")

ggplot(long.df.ex_decay, mapping = aes(x = time_min, y = conc,
    color = type)) + geom_point() + geom_smooth(method = "lm") +
    labs(title = "Example of ln linearization of first order exponential
    ↪  decay") +
```

```
xlab("time (minutes)") + ylab("ln(H2 Concentration [ppm])") +
theme(legend.position = "bottom") + scale_y_continuous(limits = c(-5,
0)) + scale_x_continuous(limits = c(0, 23))
```

Sand, M., Skeie, R.B., Sandstad, M., Krishnan, S., Myhre, G., Bryant, H., Derwent, R., *et al.*, 2023. A multi-model assessment of the Global Warming Potential of hydrogen. *Communications Earth & Environment*, **4**, 203. doi:10.1038/s43247-023-00857-8